

The image features the text "DL for ENG" in a bold, 3D, gold-colored font. The letters are floating on a vast, blue ocean with visible ripples and small waves. In the background, there are rolling blue hills or mountains under a sky filled with soft, grey clouds. The overall scene has a serene and expansive feel.

DL for ENG

LECTURE 6.1

Loss Functions and Gradients

Five lectures of models and none of them fitted. Today: where the loss comes from, how a parameter moves, and what makes the second half possible.

Remus Teodorescu

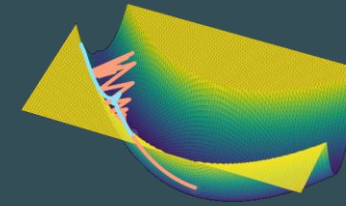
With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

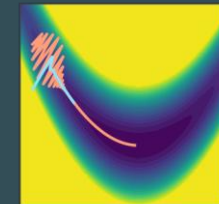
likelihood · descent · Adam · backpropagation · physics in the loss

THE TRAINING LOOP



plain SGD — zig-zags across

Adam — walks along



from above

a narrow valley —
the gradient points
across it, not along

Four lines in Ex_02 notebook 03, and you ran them without being told what any of them did. Today is the inside of the loss, the optimiser and the gradients.

the loss

derived, not memorised

the optimiser

SGD, momentum, Adam, L-BFGS

the gradients

backpropagation, once

Learning Objectives

SECTIONS, IN ORDER

- 1 **Loss function** — what a loss is and where it comes from; squared error, cross entropy, the noise model
- 2 **Optimisers** — gradient descent, momentum, Adam, L-BFGS, and the learning rate
- 3 **Backpropagation** — the chain rule reused, and what autograd records
- 4 **Initialisation** — where the parameters start, and the He and Glorot rules
- 5 **Physics in the loss** — derivatives of the input, a loss with no data, weighting two terms
- 6 **Challenges** — what goes wrong first, and what to check

WHAT WE SKIP, AND WHERE IT IS TAUGHT

capacity	the loss surface and overfitting — L4.2
the architectures	what the loss is applied to — L5
transfer, fine-tuning	starting from trained weights — L6.2
the residual in use	heat, flow, batteries, a grid — L7 onward

Section 5 is what Part 2 rests on. If the session runs short, something in the middle goes, not that.

Recap

THE TEN QUESTIONS OF L5.2 — RNN, LSTM, ATTENTION AND TRANSFORMERS

- 1** What kind of network is a recurrent network, and what function does it implement on a sequence? *Wikipedia, Recurrent neural network*
- 2** Describe its architecture: the hidden state, the unrolled network, and why the gradient over time vanishes? *Wikipedia, Recurrent neural network; Vanishing gradient problem*
- 3** What kind of network is an LSTM, and how does it differ from the plain recurrence in state, update and reach? *Wikipedia, Long short-term memory*
- 4** Describe the LSTM cell: the belt, the three gates, the candidate, and the six equations? *Wikipedia, Long short-term memory; Ex_05 notebook 04*
- 5** Given that, what is an LSTM used for, how is it sized, and why initialise the forget bias to one? *Wikipedia, Long short-term memory; Ex_05 notebook 04*
- 6** What kind of layer is attention, and what function do queries, keys and values implement? *UDL ch. 12.1 – 12.2*
- 7** Describe its architecture: the attention matrix, several heads, and why a positional encoding is needed? *UDL ch. 12.2 – 12.3; Ex_05 notebook 04*
- 8** How does attention differ from recurrence and from a dense layer, and what did adding a layer measure to? *UDL ch. 12.1; tools/deck/make_attention_fig.py*
- 9** Describe the standard transformer: embedding and position, encoder, decoder, one layer; what are BERT, GPT and ViT for? *UDL ch. 12.4 – 12.7*
- 10** A model wins by ten per cent on one run. What do you check first, and what does the residual autocorrelation add? *Ex_05 notebook 04, sections 7 and 9*

Loss Function

WHAT IT IS, WHERE IT COMES FROM, THE RECIPE

- **what** — a scalar of the parameters, small when the data are likely
- **where from** — a distribution over the measurement, your choice
- **the recipe** — choose, emit its parameters, likelihood, log, negate
- **judgement** — in the choice only; the rest is algebra

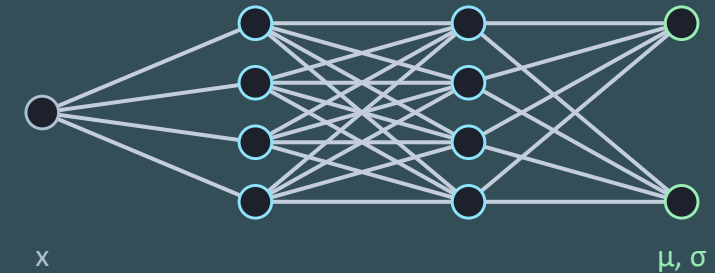
$$\hat{\theta} = \operatorname{argmin}_{\theta} - \sum_i \log P(y_i | f(x_i, \theta))$$

ϑ the parameters, P the distribution the network emits, y the measurements: minimise the negative log likelihood.

ONE RECIPE, THREE MEASUREMENTS

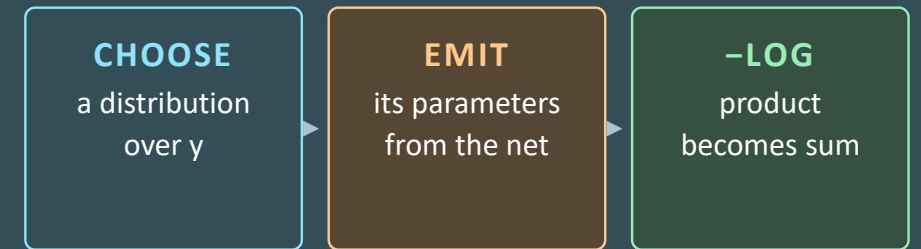
measurement	distribution	the network emits	the loss
a real number	Gaussian	the mean μ	squared error
yes or no	Bernoulli	a probability p	binary cross entropy
one of K	categorical	K probabilities	cross entropy

WHAT THE NETWORK EMITS



The output layer does not give the answer. It gives the parameters of a distribution over answers; the answer is that distribution's most likely value.

THE RECIPE



Squared Error

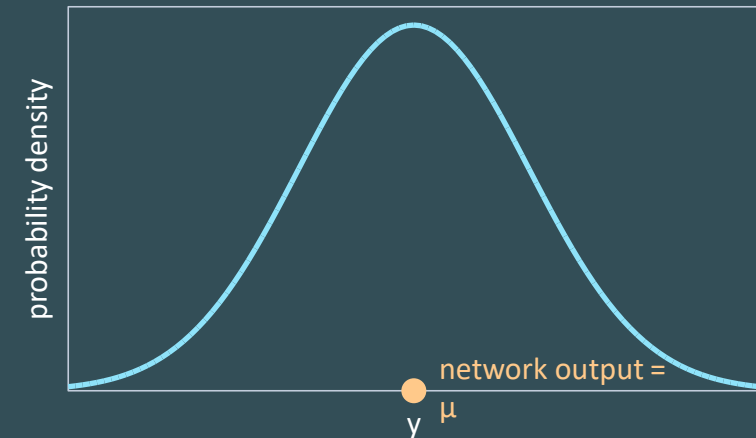
A REAL NUMBER WITH GAUSSIAN NOISE

- you measure a temperature, a stress, a voltage
- **assume** — truth plus independent Gaussian noise
- **substitute** — the exponential cancels against the logarithm
- what is left is the sum of squared errors
- **σ scales the loss and drops out** — until there are two terms

$$-\log P = \frac{1}{2\sigma^2} \sum_i (y_i - f(x_i, \theta))^2 + \text{const}$$

y_i the measurement, $f(x_i)$ the prediction, σ the noise level — constant, so it drops out

ONE MEASUREMENT, AS A DISTRIBUTION



The network places this bell over each input. Maximising its height at your measurements is the same thing as minimising the squared error.

Ex_06 notebook 01 evaluates both surfaces on a 601×601 grid and finds the same minimiser to three decimals.

Cross Entropy

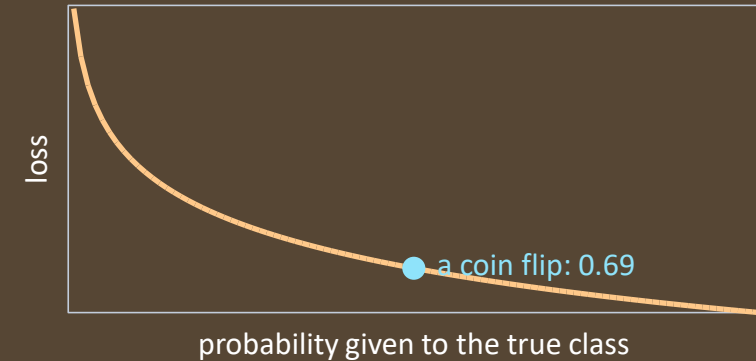
YES OR NO, AND ONE OF K CLASSES

- **yes or no** — Bernoulli; a sigmoid makes a probability
- **one of K** — categorical; a softmax makes K probabilities
- **the loss** — minus the log of the probability given to the truth
- **confident and wrong** — charged heavily; it falls out of the log
- **the check** — an untrained K-class model reports about $\ln K$

$$\mathcal{L} = - \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

$y_i \in \{0, 1\}$ the label, p_i the predicted probability of class 1

THE PENALTY, AND THE SOFTMAX



$$p_k = \frac{\exp(z_k)}{\sum_j \exp(z_j)}$$

$$\mathcal{L} = - \sum_i \log p_{y_i}$$

CrossEntropyLoss applies the softmax itself; never twice.

Noise Model

THE ASSUMPTION BEHIND SQUARED ERROR

Squared error is optimal for Gaussian noise. Real instruments produce occasional wild readings, and a Gaussian assigns them almost no probability at all.

ONE BAD READING

Because a Gaussian assigns almost no probability to a far-out value, the fit can only explain one by moving. So the model drags itself towards a single corrupt point, and the squared penalty makes that drag grow with the square of the distance.

In Ex_06 notebook 01 one bad reading in forty moves the least-squares slope by nine per cent and the intercept by thirty-five. One measurement out of forty, and not a pathological one.

The fit is still the maximum likelihood answer. It is the right answer to the wrong question.

A HEAVIER TAIL

Assume a heavier-tailed distribution — a Laplace, which says large deviations are unlikely but not absurd. Run the same recipe.

Out comes the sum of absolute errors rather than squared errors. The absolute-error fit on the same corrupted data lands within two per cent of the truth on both parameters.

Nobody looked up a robust loss. One modelling assumption changed and the recipe produced the loss that follows from it.

The cost: absolute error has no derivative at zero and optimises less smoothly.

Choosing a loss is choosing a noise model. Say which one you assumed, in the report, in one sentence.

Gradient Descent

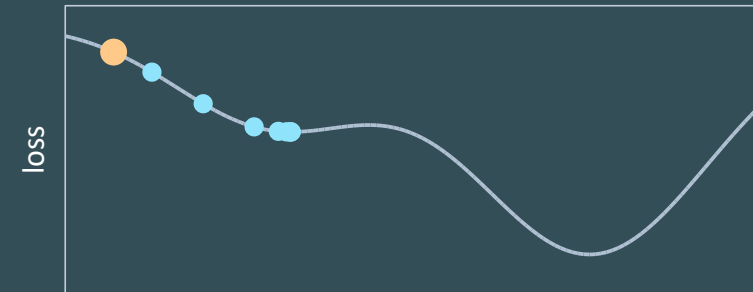
DOWNHILL IN SMALL STEPS, WITH AND WITHOUT NOISE

- the loss is a surface over the parameters — one axis each
- compute the slope where you stand, step downhill, repeat
- long strides where it is steep, short near the bottom
- **local** — it settles in the nearest basin, not the deepest
- a **minibatch** — a noisy gradient, cheaper, and the noise explores
- **batch size** — not only a memory setting

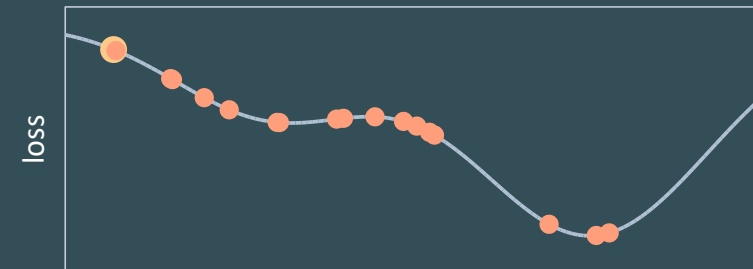
$$\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} \mathcal{L}(\theta_t)$$

θ the parameters, α the learning rate, $\nabla_{\theta} \mathcal{L}$ the gradient, t the step

PLAIN, THEN WITH NOISE



plain: nine steps, the nearest basin



one parameter

with minibatch noise: it rattles across into the deeper basin

Momentum

A RUNNING AVERAGE OF THE GRADIENT

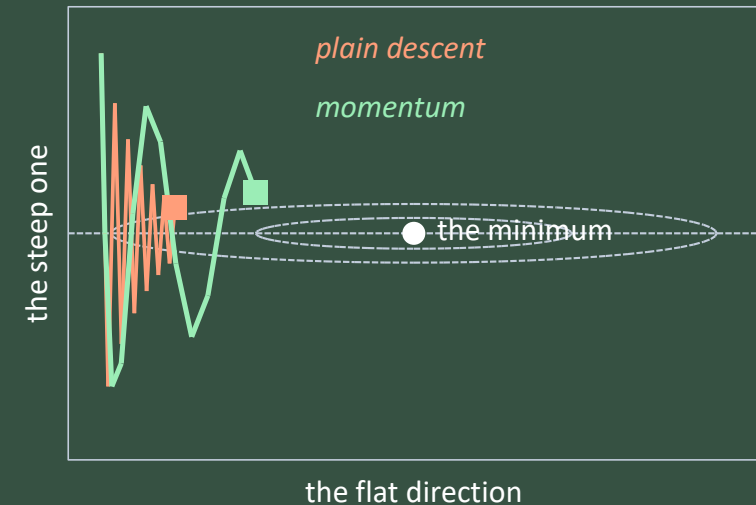
- most loss surfaces are far steeper in some directions
- in a narrow valley the gradient points across it, not along
- **average recent gradients** — the across-valley parts cancel
- a ball with mass, not a point
- **Ex_06 notebook 02** — same rate, same data, ten times lower loss

$$m_{t+1} = \beta m_t + (1 - \beta) \nabla_{\theta} \mathcal{L} \quad \theta_{t+1} = \theta_t - \alpha m_{t+1}$$

m the running average of the gradient, β how much of it survives each step

TALK · 1 MIN In the valley on the right, why does the orange path cross the valley at every step instead of walking along it, and what would a larger learning rate do to it?

A NARROW VALLEY, SEEN FROM ABOVE



Dashed rings are contours of equal loss; squares mark twelve steps. Momentum has covered more than twice the distance along the valley.

Adam

A LEARNING RATE PER PARAMETER

- **momentum fixed the direction** — Adam fixes the scale
- **two running averages** — the gradient, and its square
- **divide one by the root of the other** — small gradients still move
- **the default since 2015** — almost nobody tunes its betas
- **not monotone** — report the final loss or the best, and say which

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

m the averaged gradient, v its averaged square, ϵ stops a divide by zero

WHAT EACH PIECE ADDS

gradient the direction downhill

+ momentum averages out the zig-zag

+ squared average per-parameter scaling

+ bias correction fixes the first few steps

= Adam the default since 2015

SETTINGS

$\text{lr} = 1\text{e-}3$, $\text{betas} = (0.9, 0.999)$, $\text{eps} = 1\text{e-}8$; change the first, leave the rest.

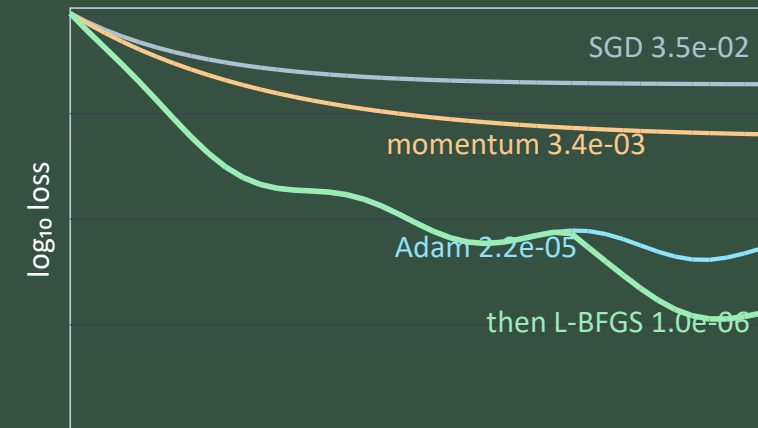
L-BFGS

A SECOND-ORDER METHOD, AND WHEN TO SWITCH

- everything so far used only the slope
- **curvature says how far to step** — better near a minimum
- L-BFGS estimates it from the gradients already seen
- **full-batch** — the same loss every call; no minibatches
- **Adam gets close, L-BFGS finishes** — the Part 2 recipe

TALK · 2 MIN In the plot on the right the cyan curve ripples and the green one drops at the hand-off. What does L-BFGS use that Adam does not, and what does that cost per step?

MEASURED IN EX_06 NOTEBOOK 02



training progress

Adam, 2000 steps	into the right basin
then L-BFGS, 60	several hundred evaluations
gain	22 × over Adam alone

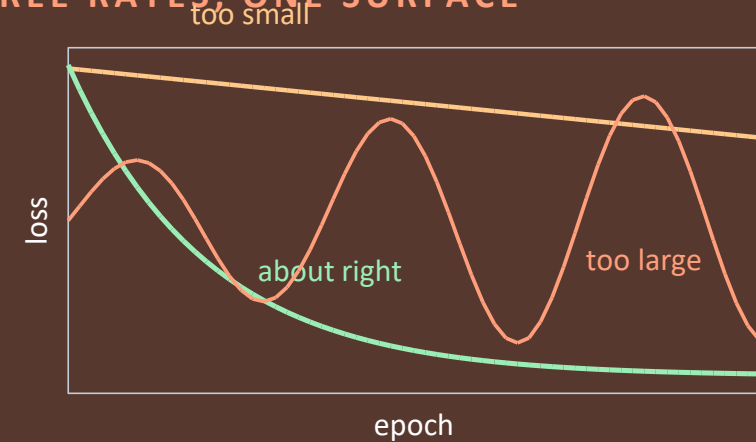
Learning Rate

THE ONE HYPERPARAMETER THAT MATTERS MOST

- **tune one number** — tune the learning rate
- **too large** — it diverges, visibly; the easy failure
- **too small** — it falls slowly for ever, and looks like a working run
- **search on a log scale** — factors of ten, not decimals
- **a decay schedule** — travel early, settle late

TALK · 2 MIN Your loss went flat after ten epochs. Which of the three curves on the right is your run, and is the learning rate too high or too low?

THREE RATES, ONE SURFACE



WHAT TO TUNE, IN ORDER

- 1 learning rate — decades, not decimals
- 2 how long you train
- 3 network size — the capacity sweep

Backpropagation

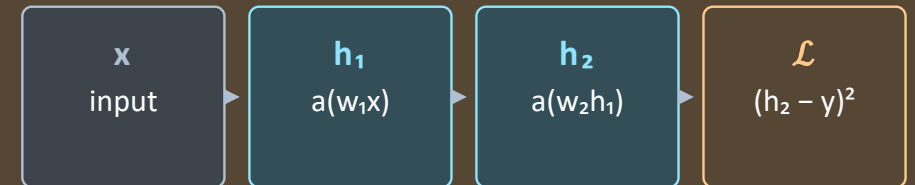
THE CHAIN RULE, IN THE ORDER THAT PERMITS REUSE

- every optimiser needs the gradient on every parameter
- forward** — compute and store every intermediate value
- backward** — from the loss, times each node's own derivative
- reuse** — every parameter shares the same partial products
- the cost** — about one extra forward pass, for all of them
- the memory** — training stores what inference does not

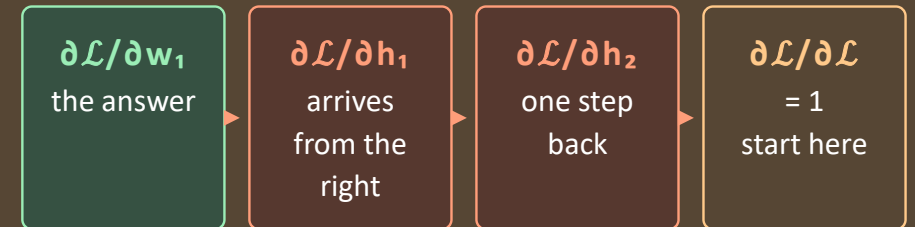
$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w_1}$$

w_1 a weight in the first layer, each factor is one layer, multiplied right to left

FORWARD, THEN BACKWARD



forward: left to right, storing x , h_1 , h_2



backward: right to left, each node times its own derivative

local × incoming this weight's gradient

Ex_02 notebook 02 by hand, vs autograd

Autograd

WHAT THE FRAMEWORK RECORDS

- PyTorch records every operation on a tensor into a graph
- **backward() walks that graph** — you never write the chain rule
- **built as the forward pass runs** — control flow as taken
- **backward() frees it** — a second call is an error
- **it connects tensors** — it does not know which are weights

FOUR CALLS THAT BITE

zero_grad() gradients add up if you forget

backward() once the graph is freed after it

no_grad() for inference — saves the memory

detach() cuts the graph deliberately

ANY TENSOR

Autograd differentiates with respect to any tensor in the recorded graph, and the input is a tensor like any other.

Initialisation

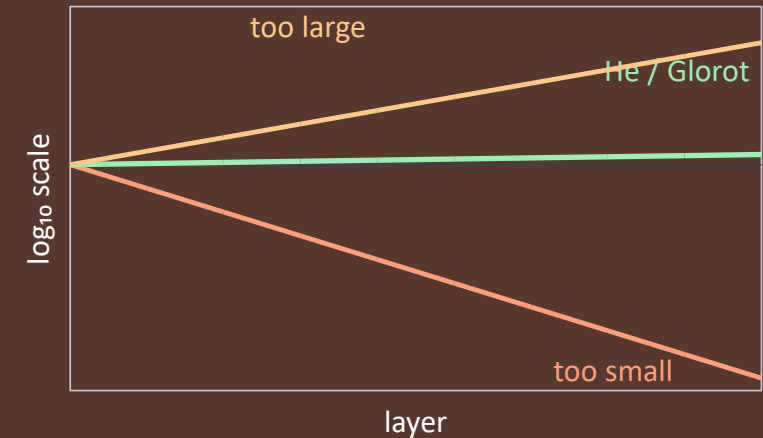
WHERE THE PARAMETERS START, AND WITH WHAT SPREAD

- **all zero** — every neuron computes the same thing, for ever
- **random, then** — but with what spread
- too large and activations grow layer by layer; too small, they vanish
- **the same for gradients on the way back** — before any training
- **the rule** — variance 1 over the fan-in; He doubles it for ReLU
- **PyTorch does this by default** — it matters in a layer you write

$$\text{Var}(w) = \frac{2}{D_{\text{in}}} \quad \text{Var}(w) = \frac{2}{D_{\text{in}} + D_{\text{out}}}$$

He, on the left, for ReLU. Glorot, on the right, for tanh — every PINN in Part 2.

ACTIVATION SCALE, TEN LAYERS



WHICH ONE, AND WHEN

ReLU	He — variance 2 / fan-in
tanh, sigmoid	Glorot — 2 / (fan-in + fan-out)

Derivatives of the Input

ONE CALL

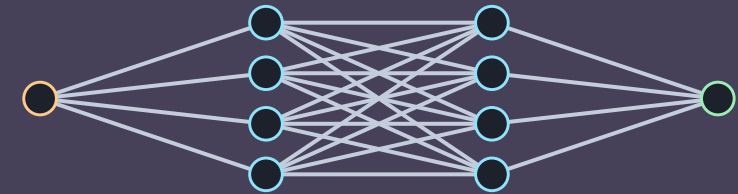
`torch.autograd.grad(u, x)` — the derivative of the network's output with respect to its input. Not its weights. Its input.

WHY IT WORKS

- backpropagation walks a graph of tensors
- the graph does not know which tensors are weights
- so ask for the derivative with respect to an input
- **a network u of position x** — du/dx , and again for d^2u/dx^2
- Ex_02 notebook 02 did this on an analytic function

$$\frac{\partial u}{\partial x}, \frac{\partial^2 u}{\partial x^2}$$

GRADIENTS GOING THE OTHER WAY



$\frac{\partial u}{\partial x}$ — the same backward sweep, one node further left u

w.r.t. weights

training — what you know

w.r.t. inputs

physics — what is next

twice

second derivatives, for a PDE

Residual Loss

A LOSS WITH NO DATA IN IT

A loss is any differentiable number you want driven to zero. Nothing in that sentence requires a measurement, a label or a target.

THE EQUATION AS THE LOSS

- **take the heat equation** — move everything to one side
- **what is left is a residual** — zero when the equation holds
- **the network is the function** — autograd gives every derivative
- **square it at sampled points, average** — a loss with no data
- every technique in this lecture applies to it unchanged

TWO LOSSES, SIDE BY SIDE

SUPERVISED

needs x, needs y,
compares them

RESIDUAL

needs x only,
compares to physics

$$\mathcal{L}_{\text{phys}} = \frac{1}{N} \sum_j \left(\frac{\partial u}{\partial t} - \alpha \frac{\partial^2 u}{\partial x^2} \right)^2$$

No y appears anywhere. The only inputs are the points at which you chose to evaluate it — chosen, not measured, and free to resample every epoch.



Weighted Losses

TWO TERMS, AND THE NUMBER BETWEEN THEM

- **in practice both** — sparse measurements and an equation
- add the two losses, with λ between them
- **sweeping λ for the best test error** — fits the test set
- **λ is the ratio of the two noise scales** — the σ from before
- **so it is measurable** — a thermocouple has a stated accuracy

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda \mathcal{L}_{\text{phys}}$$

$\mathcal{L}_{\text{data}}$ fit to measurements, $\mathcal{L}_{\text{phys}}$ the equation residual, λ what you have to choose

WHAT λ SAYS

trust the data

trust the physics

At the left the equation is ignored and you have a curve fit. At the right the measurements are ignored and you have a solver. Where you sit is a claim about your instruments.

IN THE REPORT

The value of λ , the argument for it in one sentence, and what changed when you moved it by a factor of ten. Three lines. Every Part 2 report is marked on them.

Challenges

NaN AFTER FOUR STEPS

learning rate too large, a log of zero, an exploding initialisation ·
divide the rate by ten first · one keystroke, and the cause most
of the time

THE LOSS FALLS SLOWLY FOR EVER

learning rate too small, or dead neurons from a bad initialisation
· looks like a working run · print the activation statistics on batch
one

THE LOSS DOES NOT MOVE

a missing zero_grad, an optimiser over the wrong parameters, a
detach that cut the graph · check the gradients are non-zero
before touching the model

DATA AND ENERGY EFFICIENCY

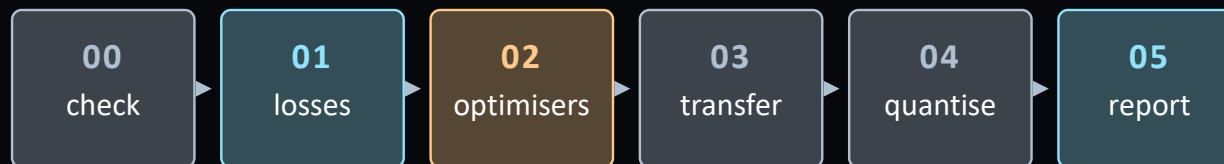
free	divide the learning rate by ten
free	check the first loss $\approx \ln K$
free	print gradient norms after backward
cheap	activation statistics on batch one
cheap	overfit five samples
real work	reconsider the noise model

The last row is the noise-model slide. A loss that assumes Gaussian noise on a sensor that spikes is dragged by three bad readings, and no optimiser setting fixes it.

Exercise 6

SIX NOTEBOOKS; 01 AND 02 ARE THIS LECTURE'S

- **00** — environment check, and the L-BFGS closure convention
- **01** — the loss recipe, by hand and then in PyTorch
- **02** — four optimisers on one problem; the measured figure
- **03** — transfer, then fine-tune; for the next lecture
- **04** — quantisation; for the next lecture
- **05** — the report: which optimiser, and why



THE QUESTIONS 01 AND 02 ANSWER

- 01** is the Gaussian negative log likelihood the squared error, to three decimals?
- 01** what does one bad reading in forty do to a least-squares fit?
- 02** four optimisers, one problem: how far apart do they land, and at what cost?

REPORT QUESTION

Why two optimisers instead of one? Answer with a number from notebook 02 and the condition under which the answer stops holding.

Key Takeaways

1 Loss function — a noise model

A loss is the negative log likelihood of the data under a distribution you chose. Gaussian noise gives squared error, Bernoulli and categorical give cross entropy, a heavier tail gives absolute error. Say which one you assumed.

2 Optimisers — direction, scale, curvature

Gradient descent steps downhill; momentum averages away the zig-zag; Adam scales each parameter; L-BFGS uses curvature to finish. Measured in notebook 02: $3.5\text{e-}02$, $3.4\text{e-}03$, $2.2\text{e-}05$, $1.0\text{e-}06$. Tune the learning rate first, in decades.

3 Backpropagation — the chain rule, reused

Forward stores every value, backward multiplies each node's derivative by what arrived from the right. All the gradients cost about one extra pass, and the recorded graph connects tensors, not weights.

4 Initialisation — keep the scale constant

Zero fails, too large saturates, too small dies, all before the first step. Variance one over the fan-in, doubled for ReLU (He), balanced for tanh (Glorot). Print the activation statistics on batch one.

5 Physics in the loss — no data needed

Autograd differentiates with respect to the input, so a PDE residual is a loss with no measurements in it. With data too, λ is the ratio of the two noise scales, measurable, and belongs in the report as a number with an argument.

Next — L6.2: what happens to a model after it has been trained.

Questions

TEN QUESTIONS FOR THE ORAL EXAMINATION

- 1 What is a loss function, where does it come from, and what is the five-step recipe? *UDL ch. 5.1 – 5.2*
- 2 What distribution does squared error assume, and what happens to that fit when a sensor spikes? *UDL ch. 5.3, 5.8; Ex_06 notebook 01*
- 3 Derive cross entropy from a categorical distribution; what should an untrained ten-class model report? *UDL ch. 5.5 – 5.7*
- 4 Why does plain gradient descent zig-zag in a narrow valley, and what does momentum change? *UDL ch. 6.1 – 6.3; Ex_06 notebook 02*
- 5 What two averages does Adam keep, what does each fix, and why is its loss not monotone? *UDL ch. 6.4; Kingma & Ba (2015)*
- 6 What does L-BFGS use that Adam does not, why is it full-batch, and when is the hand-off worth it? *GBC ch. 8.6, 4.2; Ex_06 notebook 02*
- 7 What does backpropagation compute, why does it cost one extra pass, and why does training need more memory? *UDL ch. 7.2 – 7.4*
- 8 Why does a zero initialisation fail, and what do the He and Glorot rules keep constant? *UDL ch. 7.5*
- 9 Why can autograd differentiate with respect to an input, and how does that make a loss with no data? *Ex_02 notebook 02; Part 2*
- 10 What is the weight λ between two loss terms, physically, and what must a report say about it? *Ex_06 notebook 01; Part 2 reports*

NEXT

L6.2 — Post-Training and Reinforcement Learning

Training a model from scratch is now the unusual case. Next time: starting from somebody else's weights, what post-training means for a 2026 model, reinforcement learning, and how to make a network run fast enough to steer a car.

BEFORE THE NEXT SESSION

read	UDL ch. 9.3 — transfer learning
run	Ex_06 notebooks 00 to 02
bring	your four-optimiser loss plot
think about	what λ would be for your own project

Bring the plot. The gap between the Adam curve and the Adam-then-L-BFGS curve is the figure you will redraw in every Part 2 exercise.